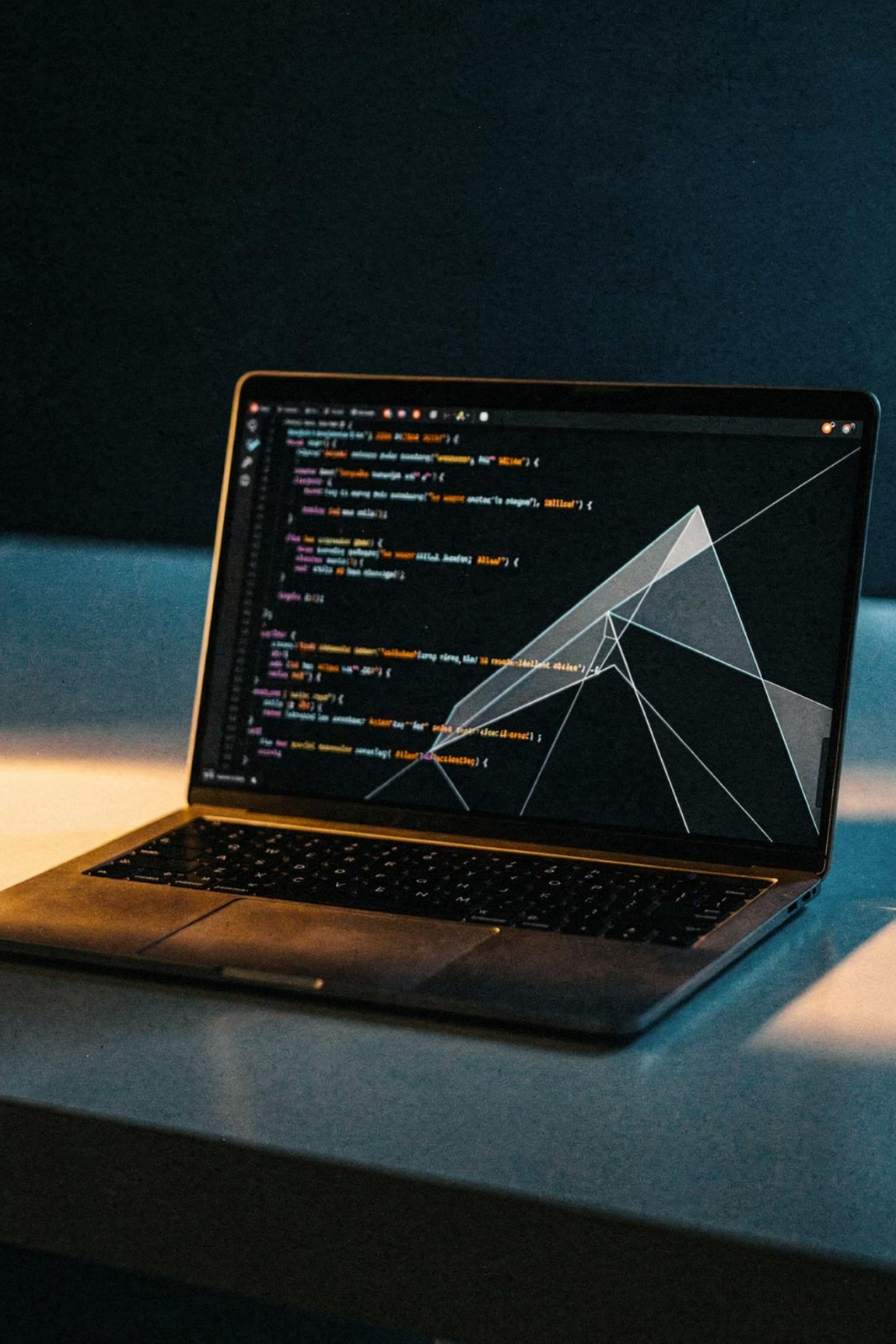


Angular Moderno

13 - Rotas do E-Commerce



13 — Rotas do E-commerce

Home • Produtos • Carrinho — Transformando sua aplicação em uma SPA com múltiplas páginas reais usando o Angular Router moderno (Standalone). Esta aula dá continuidade direta ao projeto iniciado nas aulas anteriores.

Tema da Aula

Rota Home

Criação da Home e organização das rotas existentes

Rota Produtos

Reutilização do componente `ListaProdutos` como uma página roteável.

Rota Carrinho

O componente Carrinho passa a ter sua própria URL e ciclo de vida.

An aerial photograph of a winding asphalt road that curves through a dense, green forest. The road has white lane markings and a few small vehicles are visible. The forest is lush and covers the majority of the landscape.

Objetivo da Aula

Transformar a aplicação em uma **SPA com múltiplas páginas reais** — Home, Produtos e Carrinho — utilizando o Angular Router moderno com componentes Standalone.

⚠ **Atenção ao objetivo real:** O foco desta aula NÃO é resolver arquitetura. O objetivo é criar um problema real de estado: o carrinho não será compartilhado entre rotas. Isso é intencional e será resolvido em aulas futuras.

Contexto do Projeto

O que já temos

- Lista de produtos funcional
- Carrinho implementado com Signals
- Tudo em um único componente
- Carrinho acoplado ao componente de produtos

O que está faltando

A aplicação funciona, mas **não possui navegação real**. Tudo acontece em uma única tela, sem URLs distintas para cada seção.

- ❏ O carrinho ainda está dentro do componente de produtos de forma **intencional**. Isso será refatorado futuramente.

A URL controla a tela.

Cada URL corresponde a um componente específico, e o Angular Router é o responsável por fazer essa correspondência acontecer de forma eficiente e declarativa.

/ Exibe o componente Home	/produtos Exibe a Lista de Produtos	/carrinho Exibe o Carrinho
------------------------------	--	-------------------------------

 O App não decide mais o que mostrar. A URL decide.

⊗ Verificação Obrigatória. Leia antes de continuar!

Componente Raiz: Verificação Obrigatória

O Angular precisa encontrar o componente raiz no HTML para inicializar a aplicação.

Uma divergência simples pode impedir tudo de funcionar.

✓ Verifique em `app.ts`

```
selector: 'app-root'
```

✓ Verifique em `index.html`

```
<app-root></app-root>
```

O selector declarado no componente deve ser **idêntico** à tag usada no HTML.

⊗ Se houver divergência: a aplicação **NÃO inicia**. Erro comum: `NG05104`

Estratégia Didática da Aula

01

Criar novas páginas

Começamos criando o componente `Home` com a CLI do Angular.

02

Organizar rotas reais

Configuramos o arquivo `app.routes.ts` com as 3 rotas da aplicação.

03

Separar responsabilidades

Cada componente passa a ter sua própria URL e ciclo de vida independente.

04

Testar navegação

Rodamos a aplicação e testamos o fluxo de navegação entre as páginas.

05

Quebrar o carrinho (intencionalmente)

O carrinho PARA DE FUNCIONAR.
Isso é intencional.

Parar o Servidor de Desenvolvimento

Como parar

No terminal onde o `ng serve` está rodando, pressione:

`CTRL + C`

Confirme com `S` ou `Y` se solicitado.

Por que parar?

Antes de criar novos arquivos e modificar a estrutura do projeto, é uma boa prática parar o servidor. Isso evita conflitos de compilação durante as alterações.

Criar o Componente Home

Use a CLI do Angular para gerar o componente automaticamente.

Isso cria todos os arquivos necessários (.ts, .html, .css) com a estrutura correta.

```
ng g c features/home/home
```

O que a CLI cria

- `home.ts` — lógica do componente
- `home.html` — template visual
- `home.css` — estilos do componente
- `home.spec.ts` — arquivo de testes

Importante saber

Componentes gerados pela CLI no Angular v17+ são **Standalone por padrão**. Não precisam de NgModule.

Isso está alinhado com a arquitetura moderna do curso.

Ajustar o `home.ts`

Verifique e ajuste o arquivo gerado. O componente deve estar limpo, sem imports desnecessários:

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-home',
  imports: [],
  templateUrl: './home.html',
  styleUrls: ['./home.css']
})
export class Home {}
```

 **Observação:** O array `imports: []` estará vazio por enquanto. Adicionaremos dependências conforme o componente evoluir nas próximas aulas.

Ajustar o `home.html`

Substitua o conteúdo padrão gerado pela CLI pelo template abaixo.

Mantenha simples. O objetivo é validar que a rota funciona:

```
<h1>Bem-vindo à Minha Loja</h1>  
<p>Este é o início da nossa aplicação de e-commerce.</p>
```

- ✅ Esta página será a primeira coisa que o usuário verá ao acessar a URL raiz `/`.
Nas próximas aulas ela receberá mais conteúdo.

Reutilizando o Componente de Produtos

NÃO faça isso

Criar um novo componente chamado "PaginaProdutos" ou similar.

Isso seria duplicação desnecessária de código.

Faça isso

O componente `ListaProdutos` já existe e já funciona. Agora ele será **usado como página**, associado à rota `/produtos`.

Esta é uma das grandes vantagens do Angular Router: componentes existentes podem se tornar páginas sem nenhuma modificação.

- ❏ Reutilização de componentes é um dos princípios fundamentais de uma boa arquitetura Angular.

Configurar as Rotas — `app.routes.ts`

Este é o coração da navegação. Cada objeto no array define uma URL e qual componente ela deve exibir:

```
import { Routes } from '@angular/router';
import { Home } from '../features/home/home/home';
import { ListaProdutos } from '../features/produtos/lista-produtos/lista-produtos';
import { Carrinho } from '../features/carrinho/carrinho/carrinho';

export const routes: Routes = [
  { path: '', component: Home },
  { path: 'produtos', component: ListaProdutos },
  { path: 'carrinho', component: Carrinho },
];
```

`path: ''`

URL raiz → Home

`path: 'produtos'`

Rota separada

`path: 'carrinho'`

Rota separada

❏ A ordem das rotas importa (avaliadas de cima para baixo).

Ajustar o `app.html`

Substitua o conteúdo do template raiz da aplicação pelo código abaixo.

Este é o layout base que envolve todas as páginas:

```
<h1>Minha Loja</h1>

<nav>
  <a routerLink="/">Home</a> |
  <a routerLink="/produtos">Produtos</a> |
  <a routerLink="/carrinho">Carrinho</a>
</nav>

<hr />

<router-outlet></router-outlet>
```

Entendendo o `app.html`



`routerLink`

Substitui o atributo `href` em links de navegação. A diferença crucial: `routerLink` navega sem recarregar a página (sem "reload"), preservando o estado da aplicação Angular em memória.



`router-outlet`

É o espaço reservado dinâmico onde o Angular renderiza o componente correspondente à URL atual. Pense nele como uma "janela" que troca de conteúdo conforme a rota muda.

- ❗ O `header` com a navegação ficará fixo em todas as páginas. Apenas o conteúdo dentro do `router-outlet` vai mudar.

Revisar o `app.ts`

O componente raiz precisa importar as diretivas de roteamento para que o `routerLink` e o `router-outlet` funcionem no template:

```
import { Component, signal } from '@angular/core';
import { RouterOutlet, RouterLink } from '@angular/router';

@Component({
  selector: 'app-root',
  imports: [RouterOutlet, RouterLink],
  templateUrl: './app.html',
  styleUrls: ['./app.css'],
})
export class App {
  protected readonly title = signal('meu-primeiro-app');
}
```

Importar obrigatoriamente

- `RouterOutlet`
- `RouterLink`

NÃO importar aqui

O `ListaProdutos`, `Home` e `Carrinho` **não devem** ser importados no `app.ts`. Eles são carregados automaticamente pelo Router.

Rodar a Aplicação

```
ng serve
```

Após executar o comando, aguarde a compilação. O Angular compilará todos os componentes e iniciará o servidor.

Compilando...

O Angular processa TypeScript, resolve imports e verifica tipos.

Pronto!

Acesse `http://localhost:4200` no navegador.

Hot Reload

Alterações no código são refletidas automaticamente sem reiniciar.

 Se houver erros de compilação, leia a mensagem no terminal com atenção. O Angular fornece dicas precisas sobre o que corrigir.

Testes Guiados de Navegação

Com a aplicação rodando, execute os testes abaixo em sequência e observe o comportamento:

01

Acesse /

A página **Home** deve aparecer com a mensagem de boas-vindas.

02

Clique em "Produtos"

A lista de produtos deve carregar. A URL muda para /produtos.

03

Adicione produtos ao carrinho

Adicione 2 ou 3 itens. Observe o contador no carrinho.

04

Clique em "Carrinho"

Veja o que acontece...

A photograph of a metal shopping cart in a dark, dimly lit store aisle. The cart is empty and has a small pink tag attached to its handle. The background shows shelves with various products, but they are out of focus and dark.

O Que Aconteceu com o Carrinho?

⚠️ Você adicionou produtos — mas o carrinho aparece **vazio**.

Por quê?

Ao navegar para `/carrinho`, o Angular criou uma **nova instância** do componente Carrinho. O estado anterior foi destruído.

Isso é um erro?

Não. Isso é comportamento esperado. O problema é arquitetural: o estado do carrinho vive dentro do componente, não em um lugar compartilhado.

O que aprendemos?

Componentes não compartilham estado automaticamente entre rotas. Precisamos de uma solução global.

Componentes não compartilham estado automaticamente.

Este é um dos conceitos mais importantes da arquitetura Angular. Cada vez que uma rota é ativada, o componente correspondente é **criado do zero**. Qualquer estado que existia na instância anterior é perdido.

Instâncias independentes

Cada rota tem seu próprio ciclo de vida: criação, renderização e destruição.

Estado local vs. global

Estado local (dentro do componente) é destruído ao navegar. Estado global (em um Service) persiste durante toda a sessão.

O estado local “morre” e o estado global “vive”.

Ajustes Finais e Boas Práticas

Diretivas de Rota no Template

Sempre importe `RouterLink` e `RouterOutlet` no componente onde forem usados. Em componentes Standalone, cada importação é explícita.

Estrutura de Pastas Recomendada

```
src/app/  
├─ features/  
│   ├─ home/  
│   ├─ produtos/  
│   └─ carrinho/  
└─ app.routes.ts
```

Organize por feature — cada domínio da aplicação em sua própria pasta.

✓ Uma boa estrutura de pastas facilita a manutenção e o crescimento do projeto ao longo das aulas.

Recapitulando o Fluxo Completo



O Angular Router executa este ciclo toda vez que o usuário navega para uma nova URL, seja clicando em um link `routerLink` ou digitando diretamente no navegador.

Regras Importantes do Curso

✗ NgModule

Este curso usa exclusivamente a arquitetura **Standalone** do Angular moderno. NgModule está fora do escopo.

✗ *ngIf e *ngFor

Use sempre a sintaxe de template moderna: @if e @for. As diretivas estruturais legadas não serão utilizadas.

✗ Código legado

Soluções antigas (como `forRoot()` no RouterModule) não devem ser usadas. O Router moderno já está configurado via `provideRouter()`.

📘 Seguir essas regras garante que o projeto evolua de forma consistente ao longo de todas as aulas do curso.



Resultado Esperado ao Final da Aula



3 Páginas

Home, Produtos e Carrinho com rotas funcionais



Navegação Real

Links funcionando sem reload de página



URL Dinâmica

A URL muda corretamente conforme a página



Carrinho Isolado

Estado não compartilhado — intencional e esperado

Dúvidas

?

